
Guia de instalacion y practica

Aplicacion movil Expo + API REST con Express, Prisma y SQLite

Proyecto: Agenda de personas

Objetivo

Construir una aplicacion sencilla que consulte nombres y apellidos desde un backend Express, los muestre en una FlatList de React Native y permita cargar nuevos registros en SQLite mediante Prisma.

Versiones verificadas

Herramienta	Version
Node.js	20.20.2
npm	10.9.2
Expo	57.0.23
React Native	0.86.3
Prisma / Client	6.19.0
TypeScript backend	5.9.3

1. Estructura final del proyecto

```
Practica17926/
|-- backend/
|   |-- prisma/schema.prisma
|   |-- src/index.ts
|   |-- src/initDb.ts
|   |-- .env
|   |-- package.json
|-- mobile/
|   |-- App.tsx
|   |-- package.json
|-- Especificaciones_App_Dwarf_Wiki.pdf
```

La carpeta **backend** contiene la API y la conexión con Prisma. La carpeta **mobile** contiene la pantalla Expo y las peticiones HTTP al backend.

2. Instalación de Node.js

Expo SDK 57 necesita una versión moderna de Node. Se verificó Node 20.20.2 y npm 10.9.2.

Comprobar las versiones:

```
node -v
npm -v
```

Si se usa Windows Package Manager, el comando de actualización es:

```
winget upgrade --id OpenJS.NodeJS.20 --exact --silent --accept-package-agreements --accept-source-agreements
```

Si Windows solicita permisos de administrador, instala Node LTS desde nodejs.org y abre una terminal nueva antes de continuar.

3. Crear la aplicación Expo

```
npx create-expo-app@latest mobile --template blank-typescript
cd mobile
npm install
```

Para habilitar la ejecución en navegador se instalaron los paquetes compatibles con el SDK:

```
npx expo install react-dom react-native-web @expo/metro-runtime
npx expo-doctor
```

La comprobación final debe indicar que todas las verificaciones de Expo pasaron correctamente.

4. Crear el backend Express

```
mkdir backend
cd backend
npm init -y
npm install express cors dotenv @prisma/client
npm install -D prisma typescript ts-node ts-node-dev @types/node @types/express @types/cors
```

Como ts-node no era compatible con TypeScript 7, se fijó TypeScript 5.9.3:

```
npm install -D typescript@5.9.3
```

SQLite se agregó para inicializar de forma simple la base local:

```
npm install sqlite3
```

5. Configurar Prisma y SQLite

Archivo backend/.env:

```
DATABASE_URL="file:./dev.db"
```

Archivo backend/prisma/schema.prisma:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Person {
  id          Int      @id @default(autoincrement())
  firstName   String
  lastName    String
  arrivalDate DateTime?
  isWorking   Boolean @default(false)
  createdAt   DateTime @default(now())
}
```

Generar el cliente de Prisma e inicializar las tablas:

```
npx prisma generate
npm run db:init
```

En este entorno, **prisma db push** presenta un error vacío del Schema Engine en Windows. Por eso **db:init** crea las tablas SQLite una sola vez, mientras las consultas y escrituras de la API siguen realizándose con Prisma Client.

6. Scripts del backend

```
"dev": "ts-node-dev --respawn --transpile-only src/index.ts"
"build": "tsc"
"start": "node dist/index.js"
"prisma:generate": "prisma generate"
"prisma:migrate": "prisma migrate dev"
"db:init": "ts-node src/initDb.ts"
```

Comandos de verificación:

```
npx tsc --noEmit
npm run build
```

7. API REST de personas

La API se encuentra en **backend/src/index.ts**. Usa Express para recibir peticiones y Prisma Client para leer y crear personas.

Obtener la lista:

GET `http://localhost:3000/api/personas`

Crear una persona:

POST `http://localhost:3000/api/personas`
Content-Type: `application/json`

```
{
  "firstName": "Mina",
  "lastName": "Miner"
}
```

El backend valida que ambos campos existan y responde con el registro creado, incluyendo su id y fecha.

8. Aplicacion Expo y FlatList

En **mobile/App.tsx** se define la interfaz TypeScript **Person**, se consulta la API dentro de **useEffect** y se muestran los datos con **FlatList**.

```
const API_URL = 'http://localhost:3000';

async function loadPeople() {
  const response = await fetch(`${API_URL}/api/personas`);
  setPeople(await response.json());
}
```

Para cargar una persona se usa una peticion POST. Cuando responde correctamente, se limpian los inputs y se vuelve a ejecutar loadPeople para refrescar la lista.

9. Ejecutar el proyecto

Terminal 1: iniciar el backend

```
cd D:\Seshat\Practica17926\backend
npm run dev
```

Terminal 2: iniciar Expo

```
cd D:\Seshat\Practica17926\mobile
npx expo start
```

En la terminal de Expo, presiona **w** para abrir la aplicacion en Web.

10. Probar desde un celular fisico

localhost apunta al dispositivo que ejecuta la aplicacion. En un celular, localhost es el propio celular, no la PC. Cambia API_URL por la IP local de tu computadora, por ejemplo:

```
const API_URL = 'http://192.168.1.25:3000';
```

La PC y el celular deben estar en la misma red Wi-Fi y el firewall debe permitir conexiones al puerto 3000.

11. Comprobaciones rapidas

```
# Salud de la API
Invoke-RestMethod http://localhost:3000/api/health

# Obtener personas
Invoke-RestMethod http://localhost:3000/api/personas

# Abrir la aplicacion Web
npx expo start --web
```

Resultado esperado: la API responde JSON, la pantalla muestra personas iniciales y una persona nueva aparece inmediatamente despues de presionar Agregar persona.

12. Bitacora completa de comandos

Esta es la secuencia ordenada de comandos usada para preparar el proyecto desde Windows PowerShell. Ejecuta cada bloque desde la carpeta indicada. Los comandos cd cambian de carpeta y por eso el orden importa.

12.1. Comprobar Node y npm

```
cd D:\Seshat\Practica17926
node -v
npm -v
```

Salida verificada: Node.js 20.20.2 y npm 10.9.2. Node ejecuta JavaScript en el backend y npm instala dependencias.

12.2. Crear y preparar Expo

```
npx create-expo-app@latest mobile --template blank-typescript
cd mobile
npm install
npx expo install react-dom react-native-web @expo/metro-runtime
npx expo-doctor
```

npx ejecuta herramientas de npm. expo install elige versiones compatibles con el SDK de Expo. expo-doctor revisa dependencias y configuracion.

12.3. Crear y preparar Express

```
cd D:\Seshat\Practica17926
mkdir backend
cd backend
npm init -y
npm install express cors dotenv @prisma/client
npm install -D prisma typescript ts-node ts-node-dev @types/node @types/express @types/cors
npm install -D typescript@5.9.3
npm install sqlite3
```

Las dependencias normales se necesitan cuando la API se ejecuta. Las de desarrollo ayudan a compilar, ejecutar TypeScript y tener tipos en el editor.

12.4. Generar Prisma, inicializar SQLite y verificar TypeScript

```
npx prisma generate
npm run db:init
npx tsc --noEmit
npm run build
```

prisma generate crea Prisma Client. db:init prepara la tabla y conserva datos existentes. tsc --noEmit busca errores sin crear archivos. build genera dist para produccion.

12.5. Levantar los dos procesos

```
# Terminal 1
cd D:\Seshat\Practica17926\backend
npm run dev
```

```
# Terminal 2
cd D:\Seshat\Practica17926\mobile
npx expo start
```

```
# Expo Web, si se quiere abrir directamente en navegador
npx expo start --web
```

13. Probar la API sin abrir la aplicacion

Antes de revisar la interfaz, se puede probar el backend directamente desde PowerShell. Esto ayuda a separar un problema de Express de un problema de Expo.

13.1. Salud del servidor

```
Invoke-RestMethod http://localhost:3000/api/health
```

```
{  
  "ok": true  
}
```

13.2. Leer personas con GET

```
Invoke-RestMethod http://localhost:3000/api/personas
```

13.3. Crear una persona con POST

```
$body = @{  
  firstName = "Mina"  
  lastName = "Miner"  
  arrivalDate = "2026-09-17"  
  isWorking = $true  
} | ConvertTo-Json
```

```
Invoke-RestMethod http://localhost:3000/api/personas -Method Post -ContentType "application/json" -Body $body
```

13.4. Cambiar solo el estado con PATCH

```
$body = @{ isWorking = $false } | ConvertTo-Json
```

```
Invoke-RestMethod http://localhost:3000/api/personas/1 -Method Patch -ContentType "application/json" -Body $body
```

GET consulta, POST crea y PATCH modifica solo los campos enviados. El id del ejemplo debe existir; se puede reemplazar 1 por el id que devuelva POST.

14. Que hace cada archivo

Archivo	Responsabilidad
backend/src/index.ts	Crea Express, registra middlewares, define GET/POST/PATCH y arranca el servidor.
backend/src/initDb.ts	Crea o adapta la tabla Person en SQLite y agrega datos iniciales si la base esta vacia.
backend/prisma/schema.prisma	Declara los modelos y tipos que Prisma usa para comunicarse con SQLite.
backend/.env	Guarda DATABASE_URL sin mezclar configuracion con el codigo fuente.
mobile/App.tsx	Renderiza inputs, Switch y FlatList; llama a GET, POST y PATCH mediante fetch.
package.json	Guarda dependencias y scripts como dev, build, db:init y start.

14.1. Modelo actual de Person

```
id          -> identificador numerico
firstName   -> nombre
lastName    -> apellido
arrivalDate -> fecha de llegada al fuerte
isWorking   -> true/false, estado laboral
createdAt   -> fecha automatica de alta
```

La respuesta JSON de la API conserva estos nombres para que el tipo Person de App.tsx y el modelo Prisma describan el mismo contrato.

15. Errores habituales y solucion

La Web no aparece en Expo: instalar react-dom, react-native-web y @expo/metro-runtime; luego ejecutar npx expo-doctor y abrir con npx expo start --web.

Network request failed: comprobar que backend esta ejecutandose en el puerto 3000. En celular, reemplazar localhost por la IP local de la PC y revisar firewall y Wi-Fi.

Cannot find module o errores de tipos: ejecutar npm install dentro de la carpeta correcta. Backend y mobile tienen package.json separados.

Prisma no encuentra el cliente: ejecutar npx prisma generate dentro de backend. Si la tabla no existe, ejecutar npm run db:init.

PATCH no actualiza: comprobar que el id exista y que el body tenga al menos un campo. Para el Switch, enviar { isWorking: true } o { isWorking: false }.

16. Checklist final

```
[ ] node -v y npm -v muestran las versiones esperadas
[ ] npm run dev deja la API escuchando en el puerto 3000
[ ] /api/health devuelve { ok: true }
[ ] GET carga personas en FlatList
[ ] POST agrega nombre, apellido y fecha
[ ] PATCH cambia isWorking
[ ] La app se puede abrir con npx expo start --web
[ ] En celular, API_URL usa la IP de la PC
```

Flujo completo: interfaz Expo -> petición HTTP -> ruta Express -> Prisma Client -> SQLite -> respuesta JSON -> FlatList actualizada.